Name: Dan Sadeh | **ID:** 314631763

GitHub Link: https://github.com/DanSadeh/fNIRS-Signal-Processing

Goal: To extract relative concentration changes of oxygenated (ΔHbO) and deoxygenated (ΔHbR) hemoglobin from raw fNIRS intensity data, and to isolate the physiological heartbeat from system noise using frequency domain analysis.

Theoretical Background: Neural activity demands oxygen. By shining near-infrared light into the brain and measuring its attenuation, we track these hemodynamic changes. Because light scatters through biological tissue, we use the Modified Beer-Lambert Law (MBLL). The raw data collected by the machine is unitless light intensity. To convert this exponential light decay into a linear scale that we can solve algebraically, we take the base-10 logarithm (log_{10}). The final output gives us the physical concentration changes in microMolar (µM).

Equations:

Optical Density: $OD = log_{10}\left(\frac{I_0}{I_t}\right)$

Code: We calculate this using log10(intensityZeroL1 ./ intensityTimeL1).

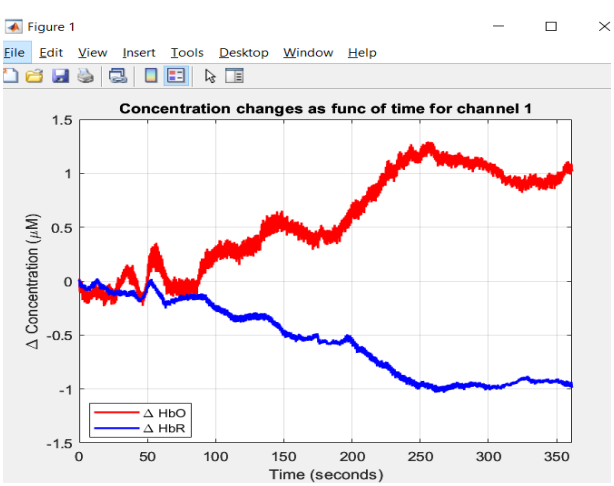
Effective Pathlength: $L_{\{eff\}} = SDS \cdot DPF \cdot relDPF$

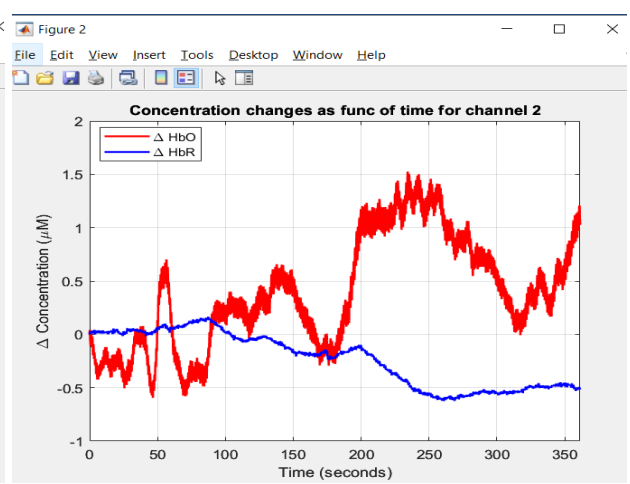
Code: We multiply the physical source-detector separation by the tissue-specific base DPF and the wavelength-dependent relative coefficient (leffL1 = sourceDetectorSep * baseDPF * relDPF1).

Concentration Change: $\Delta C = \varepsilon^{\{-1\}} \cdot \frac{OD}{L_{\{eff\}}}$

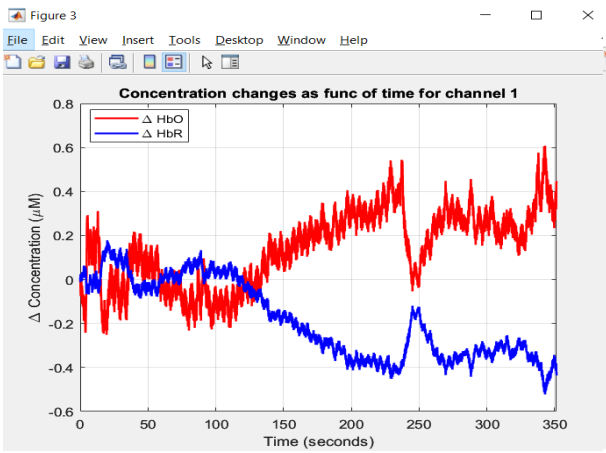
Question 2 - Time Domain Analysis:

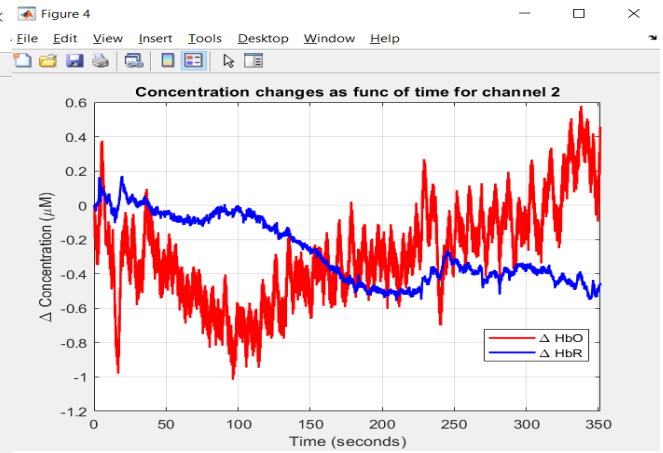
dataFile1 = '.\FN_032_V1_Postdose1_Nback.mat';





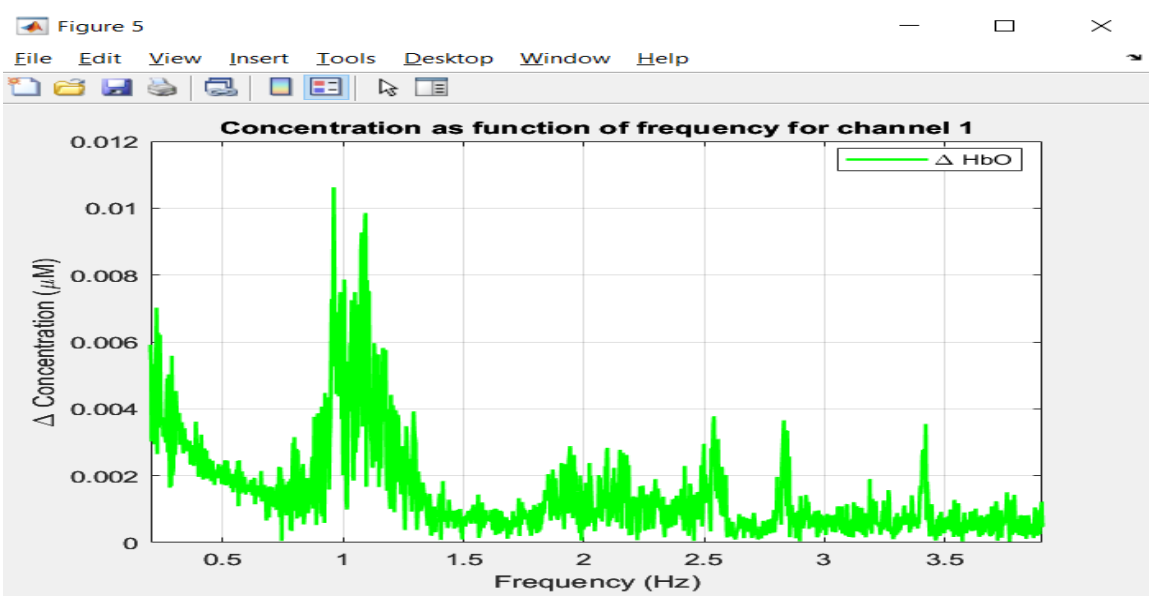
dataFile2 = '.\FN_031_V2_Postdose2_Nback.mat';





Analysis: The MBLL successfully converted raw light intensity into concentration changes. By inverting the extinction coefficient matrix, the algorithm successfully "unmixed" the overlapping light absorption spectra of the two chromophores. The plots show ΔHbO and ΔHbR fluctuating inversely in the microMolar (μM) range over time. This makes physical sense, as an influx of fresh, oxygenated blood naturally displaces the oxygen-depleted blood during a standard hemodynamic response.

Question 3 - Frequency Domain Analysis & SNR:




```
Heartbeat Signal Strength: 1.0622e-08
Baseline Noise Level:      7.5488e-10
Heart rate (BPM):          57.6182
Final SNR:                 14.0717
```

Analysis: We transformed the time-domain signal into the frequency domain to isolate the cardiac cycle. We analyzed only the ΔHbO signal because the physical shockwave of the heartbeat is carried by fresh arterial blood; by the time blood becomes deoxygenated, the physical pulse is heavily damped. After normalizing the Fourier transform to preserve true physical amplitude, we mathematically removed the 0 Hz DC offset. We also plotted the graph starting at 0.2 Hz to intentionally hide massive, slow physiological artifacts (like respiration and Mayer blood-pressure waves) that would otherwise skew the visual scale. The algorithm successfully isolated a distinct cardiac peak at a resting heart rate of **57.6182 BPM**. With an SNR of **14.07**, the physiological heartbeat is proven to be vastly stronger than the high-frequency (> 2.5 Hz) hardware noise.

File: EX1__314631763.m

Matlab

```
% Dan Sadeh 314631763

clear;
clc;
close all;

%% Calculation segment - Initialization and Constants
% Defining hardware and physiological constants upfront to avoid magic numbers
SOURCE_DETECTOR_SEP = 3;
HEARTBEAT_LOW_HZ = 0.8;
HEARTBEAT_HIGH_HZ = 1.5;
NOISE_FLOOR_HZ = 2.5;
MICROMOLAR_CONV = 1e6;
SECONDS_PER_MINUTE = 60;

% Defining inputs
dataFile1 = '.\FN_032_V1_Postdose1_Nback.mat';
dataFile2 = '.\FN_031_V2_Postdose2_Nback.mat';
tissueType = 'adult_head';
plotChannelIdx = [1, 2];
extinctionCoefficientsFile = '.\ExtinctionCoefficientsData.csv';
DPFperTissueFile = '.\DPFperTissue.txt';
relDPFfile = '.\RelativeDPFCoefficients.csv';

%% Calculation segment - Time Domain Processing (Question 2)
% Applying the MBLL to both recording sessions to extract relative concentrations
[dHbR_1, dHbO_1, fig_1] = CalcNIRS(dataFile1, SOURCE_DETECTOR_SEP, tissueType, plotChannelIdx, ...
    extinctionCoefficientsFile, DPFperTissueFile, relDPFfile);

[dHbR_2, dHbO_2, fig_2] = CalcNIRS(dataFile2, SOURCE_DETECTOR_SEP, tissueType, plotChannelIdx, ...
    extinctionCoefficientsFile, DPFperTissueFile, relDPFfile);

%% Calculation segment - Frequency Domain Transformation (Question 3)
% Extracting the time vector to determine the hardware sampling rate for the FFT
```

```
dataStruct1 = load(dataFile1);
timeVec = dataStruct1.t;
samplingFreq = 1 / mean(diff(timeVec));
numSamples = length(timeVec);

% Subtracting the mathematical mean eliminates the 0Hz DC offset, preventing
% the baseline concentration from ruining the amplitude scale of our graph.
centeredSignalChannel1 = dHbO_1(:, 1) - mean(dHbO_1(:, 1));

% Executing the discrete Fourier transform
fftResultHbO = fft(centeredSignalChannel1);

% Normalizing by the total sample count ensures the Y-axis reflects true physical
% concentration changes rather than arbitrary summation energy.
fftAbsHbO = abs(fftResultHbO / numSamples);

% Slicing the array to discard negative frequencies, as they are mathematical
% mirrors and hold no biological relevance.
fftFinalHbO = fftAbsHbO(1:floor(numSamples/2) + 1);

% Multiplying by 2 conserves the physical energy lost from discarding the negative frequency bins.
fftFinalHbO(2:end-1) = 2 * fftFinalHbO(2:end-1);

% Constructing the physical X-axis by multiplying bin indices by the frequency resolution
freqAxis = (0:floor(numSamples/2)) * (samplingFreq / numSamples);

%% Plot 1 segment - FFT Spectrum Visualization
figure;
% Magnifying signal to microMolar to view biological changes clearly
plot(freqAxis, fftFinalHbO * MICROMOLAR_CONV, 'g', 'LineWidth', 1.5);
xlim([0.2, max(freqAxis)]); % Starting from 0.2Hz to exclude huge Mayer wave/respiration noise
title('Concentration as function of frequency for channel 1');
xlabel('Frequency (Hz)')
ylabel('\Delta Concentration (\muM)');
legend('\Delta HbO', 'location', 'best');
grid on;

%% Calculation segment - SNR & Heart Rate
% Isolating the frequency bins that correspond exclusively to the biological cardiac cycle
heartBeatMask = (freqAxis >= HEARTBEAT_LOW_HZ & freqAxis <= HEARTBEAT_HIGH_HZ);
[heartBeatStrength, maxIdx] = max(fftFinalHbO(heartBeatMask));

% Defining the baseline hardware noise floor
noiseMask = (freqAxis >= NOISE_FLOOR_HZ);
noiseAvg = mean(fftFinalHbO(noiseMask));

% Final ratio calculation
SNR = (heartBeatStrength / noiseAvg);

% Extracting exact frequency to convert to Beats Per Minute (BPM)
maskedFreqs = freqAxis(heartBeatMask);
heartBeatFreq = maskedFreqs(maxIdx);
bpm = heartBeatFreq * SECONDS_PER_MINUTE;

fprintf('Heartbeat Signal Strength: %.4e\n', heartBeatStrength);
fprintf('Baseline Noise Level:      %.4e\n', noiseAvg);
```

```
fprintf('Heart rate (BPM):        %.4f\n', bpm);
fprintf('Final SNR:               %.4f\n', SNR);
```

File: CalcNIRS.m

Matlab

```
% Dan Sadeh 314631763
function [dHbR, dHbO, fig] = CalcNIRS(dataFile, sourceDetectorSep, tissueType, plotChannelIdx, ...
    extinctionCoefficientsFile, DPFperTissueFile, relDPFfile)

    % ----------------------------------------------------------------------
    % Description : The function calculates and plots HbR and HbO concentration
    %               changes using the Modified Beer-Lambert Law.
    % Assumptions : The data matrix 'd' contains exactly 40 columns (20 channels
    %               per wavelength). sourceDetectorSep is positive.
    % Inputs :
    %   dataFile - .mat file with intensity data (d), time (t), and Lambda.
    %   sourceDetectorSep - Source-Detector Separation distance in cm.
    %   tissueType - String matching a row in the DPF text file.
    %   plotChannelIdx - Vector indicating channels to plot.
    %   extinctionCoefficientsFile - .csv file with extinction parameters.
    %   DPFperTissueFile - .txt file with baseline DPF multipliers.
    %   relDPFfile - .csv file for wavelength-specific DPF adjustment.
    % Outputs :
    %   dHbR - HbR concentration change matrix (time x 20).
    %   dHbO - HbO concentration change matrix (time x 20).
    %   fig  - Handle to generated figures. Empty if no plots requested.
    % ----------------------------------------------------------------------

    %% Calculation segment - Input Validation and Constants
    fig = [];

    % Defining hardware architecture constants to eliminate magic numbers
    NUM_CHANNELS = 20;
    NUM_COLUMNS = 40;
    MICROMOLAR_CONV = 1e6;

    if nargin < 3
        error('You must provide at least dataFile, sourceDetectorSep, and tissueType.');
    end

    if ~(ischar(dataFile) || isstring(dataFile)) || ~exist(dataFile, 'file')
        error('dataFile must be a valid text string pointing to an existing file.');
    end

    if ~isnumeric(sourceDetectorSep) || ~isscalar(sourceDetectorSep) || sourceDetectorSep <= 0
        error('sourceDetectorSep must be a single positive number.');
    end

    if nargin < 4 || isempty(plotChannelIdx)
        plotChannelIdx = [];
    elseif ~isnumeric(plotChannelIdx) || ~isequal(round(plotChannelIdx), plotChannelIdx) || any(plotChannelIdx < 1)
|| any(plotChannelIdx > NUM_CHANNELS)
        error('plotChannelIdx must contain only whole numbers between 1 and %d.', NUM_CHANNELS);
```

```
    end

    if nargin < 5 || isempty(extinctionCoefficientsFile); extinctionCoefficientsFile = 'ExtinctionCoefficientsData.csv';
end
    if nargin < 6 || isempty(DPFperTissueFile); DPFperTissueFile = 'DPFperTissue.txt'; end
    if nargin < 7 || isempty(relDPFfile); relDPFfile = 'RelativeDPFCoefficients.csv'; end

    %% Calculation segment - Data Extraction
    dataStruct = load(dataFile);

    if ~isfield(dataStruct, 'd') || ~isfield(dataStruct, 't') || ~isfield(dataStruct, 'SD') || ~isfield(dataStruct.SD, 'Lambda')
        error('Loaded file is missing required fields (d, t, SD, or Lambda).');
    end

    if size(dataStruct.d, 2) ~= NUM_COLUMNS
        error('Expected the data matrix to have exactly %d columns.', NUM_COLUMNS);
    end

    intensityMat = dataStruct.d;
    timeVec = dataStruct.t;
    lambda1 = dataStruct.SD.Lambda(1);
    lambda2 = dataStruct.SD.Lambda(2);

    extCoeffTable = readtable(extinctionCoefficientsFile);
    dpfTissueTable = readtable(DPFperTissueFile);
    relDpfTable = readtable(relDPFfile);

    %% Calculation segment - MBLL Pre-calculations
    % Extracting the tissue-specific pathlength multiplier
    tissueIdx = find(strcmp(dpfTissueTable.Tissue, tissueType));
    if isempty(tissueIdx)
        error('Tissue type not found in DPF table.');
    end
    baseDPF = dpfTissueTable.DPF(tissueIdx);

    % Adjusting pathlength based on photon scattering variance at different wavelengths
    lambda1Idx = find(relDpfTable.wavelength == lambda1);
    lambda2Idx = find(relDpfTable.wavelength == lambda2);
    relDPF1 = relDpfTable.relDPFcoeff(lambda1Idx);
    relDPF2 = relDpfTable.relDPFcoeff(lambda2Idx);

    % Calculating Effective Pathlength (Leff)
    leffL1 = sourceDetectorSep * baseDPF * relDPF1;
    leffL2 = sourceDetectorSep * baseDPF * relDPF2;

    % Defining the inverse (1/Leff) here optimizes loop performance later
    invLeffL1 = 1 / leffL1;
    invLeffL2 = 1 / leffL2;

    % Building and immediately inverting the Extinction Coefficient Matrix (Epsilon)
    epsIdxL1 = find(extCoeffTable.wavelength == lambda1);
    epsIdxL2 = find(extCoeffTable.wavelength == lambda2);

    epsMatrix = [extCoeffTable.HbO2(epsIdxL1), extCoeffTable.HHb(epsIdxL1);
                 extCoeffTable.HbO2(epsIdxL2), extCoeffTable.HHb(epsIdxL2)];
    invEpsMatrix = inv(epsMatrix);
```

```
    %% Calculation segment - Core MBLL Matrix Multiplication
    % Preallocating arrays limits dynamic memory expansion inside the loop,
    % significantly reducing computational overhead.
    dHbO = zeros(length(timeVec), NUM_CHANNELS);
    dHbR = zeros(length(timeVec), NUM_CHANNELS);

    for channelIdx = 1:NUM_CHANNELS
        % Isolating baseline and continuous data for the specific channel
        intensityZeroL1 = intensityMat(1, channelIdx);
        intensityTimeL1 = intensityMat(:, channelIdx);

        intensityZeroL2 = intensityMat(1, NUM_CHANNELS + channelIdx);
        intensityTimeL2 = intensityMat(:, NUM_CHANNELS + channelIdx);

        % Calculating Optical Density using base 10 logarithmic attenuation
        opticalDensityL1 = log10(intensityZeroL1 ./ intensityTimeL1);
        opticalDensityL2 = log10(intensityZeroL2 ./ intensityTimeL2);

        % Normalizing optical density by the photon's travel distance
        scaledL1 = opticalDensityL1 .* invLeffL1;
        scaledL2 = opticalDensityL2 .* invLeffL2;

        % Solving the linear system to extract distinct HbO/HbR chromophore concentrations
        rightSideMatrix = [scaledL1, scaledL2]';
        concentrations = invEpsMatrix * rightSideMatrix;

        dHbO(:, channelIdx) = concentrations(1, :)';
        dHbR(:, channelIdx) = concentrations(2, :)';
    end

    %% Plot segment - Data Visualization
    if isempty(plotChannelIdx)
        disp('No channels were given so no plot is generated.');
    else
        for ii = 1:length(plotChannelIdx)
            channelToPlot = plotChannelIdx(ii);
            fig(ii) = figure;

            % Multiplying by the conversion constant brings the values into a readable microMolar integer range
            plot(timeVec, dHbO(:, channelToPlot) * MICROMOLAR_CONV, 'r', 'LineWidth', 1.5);
            hold on;
            plot(timeVec, dHbR(:, channelToPlot) * MICROMOLAR_CONV, 'b', 'LineWidth', 1.5);

            xlim([0, max(timeVec)]);
            title(sprintf('Concentration changes as func of time for channel %d', channelToPlot));
            xlabel('Time (seconds)');
            ylabel('\Delta Concentration (\muM)');
            legend('\Delta HbO', '\Delta HbR', 'Location', 'best');
            grid on;
            hold off;
        end
    end
end
```